LAB 3:

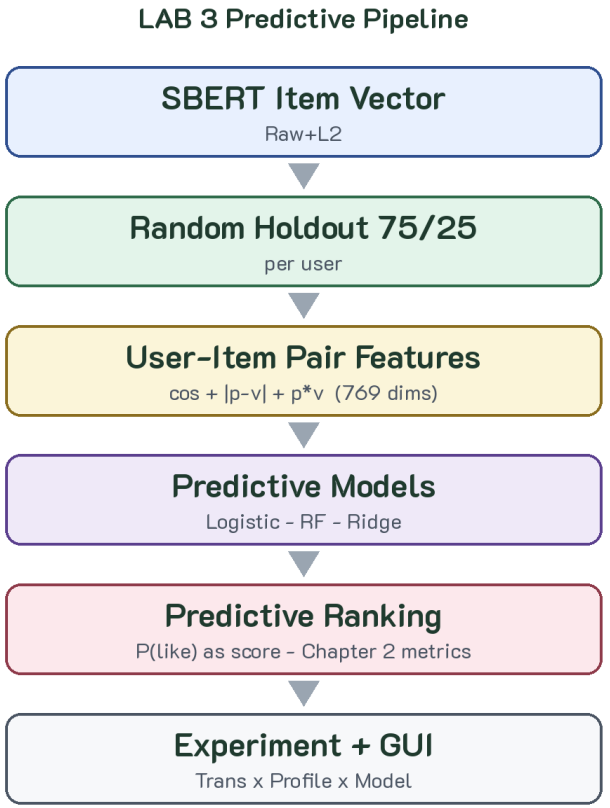
End-to-End Predictive

จากโปรไฟล์สู่โมเดลทำนาย

204729 Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

End-to-End Predictive Pipeline



อธิบายแนวคิดของ Source Code ทีละ Part

Data - Vector - Profile - Pair Features - Model - Ranking - Evaluation

COMPUTER SCIENCE
Chiang Mai University

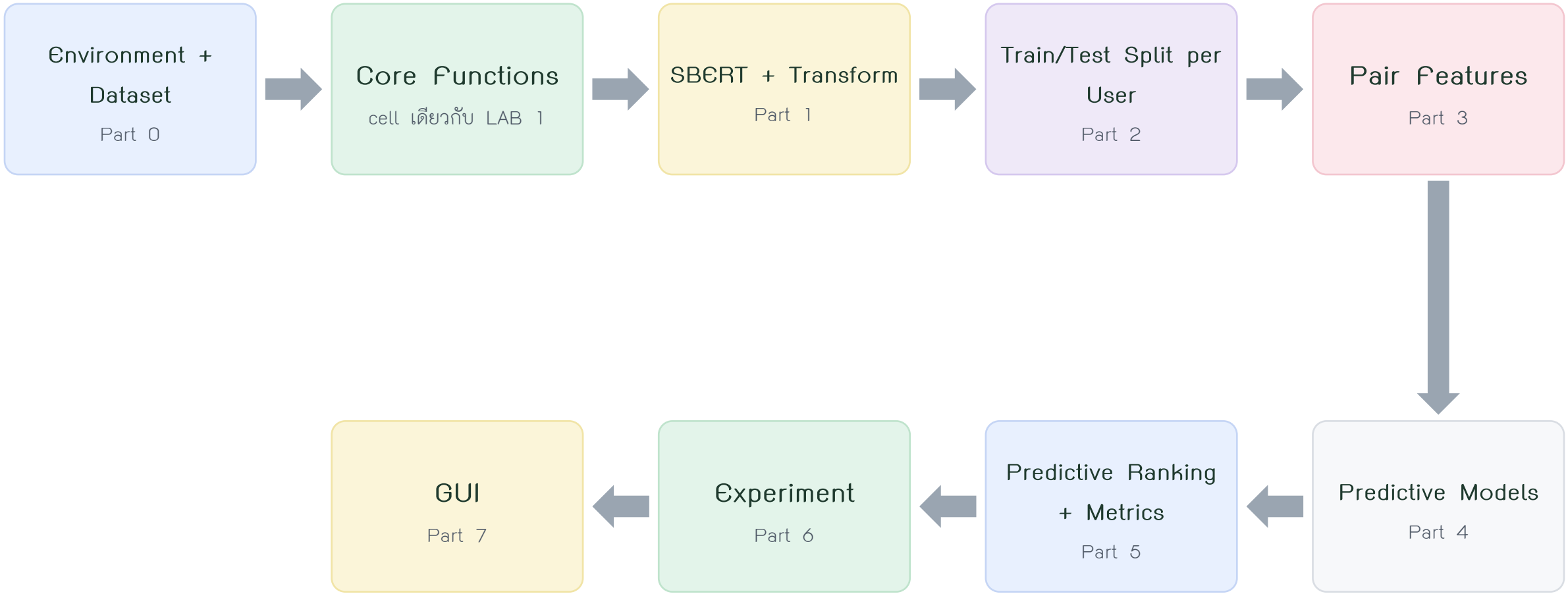
LAB 3 ต่อยอดจาก LAB 2 อย่างไร

ภาพรวม

- Part 0 และ Core Functions เหมือน LAB 1-2 ; LAB 3 ใช้ `get_profile`, `transform_vectors` และ metric เดิม แต่ **ไม่ใช้** `evaluate_config` เพราะเปลี่ยนวิธีจัดอันดับ
- Pipeline: Data ⟶ Item Vector (SBERT) ⟶ Transformation ⟶ User Profile ⟶ User-Item Pair Features ⟶ Predictive Model ⟶ Ranking ⟶ Evaluation
- Pair features = `[cos(p, v), |p - v|, p ⊙ v]` ⟶ 1 + 384 + 384 = 769 มิติ ต่อคู่ (user, item) — โมเดลเรียนรู้ว่ามิติใดของ SBERT ที่ “ตรงกับโปรไฟล์” จึงสำคัญ
- **สองงาน**: Relevance Classification (Logistic Regression, Random Forest ⟶ `predict_proba`) และ Rating Regression (Ridge, Random Forest ⟶ MAE/RMSE)
- **Split เปลี่ยนเป็น Random Holdout 75/25 ต่อผู้ใช้** (ไม่ใช่ LOO) ⟶ test มี relevant หลายเรื่องต่อคน ⟶ Precision/Recall/MAP มีความหมายมากขึ้น
- Part 5 ใช้ probability ของ classifier จัดอันดับ candidate แล้ววัดด้วย metric บทที่ 2 ; Part 6 ทดลอง Transformation × Profile × Model ; Part 7 GUI แสดง `P(like)`

จาก “เรียงตาม cosine” ⟶ “เรียนรู้ฟังก์ชันจาก feature ของคู่ (user, item)”

โครงสร้าง Notebook LAB 3 (Part 0 → Part 7)



Part 3–5 คือส่วนใหม่ทั้งหมด

แผนที่ของ Source Code LAB 3

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	สิ่งใหม่
0 + Core	Dataset + ฟังก์ชันกลาง	`items`, `ratings`, `get_profile`, `transform_vectors`, metrics	เหมือน LAB 1–2
1	SBERT item vector	`X_raw` (12 × 384), `X = transform_vectors(X_raw, "Raw+L2")`	ใช้ SBERT เป็นหลัก
2	Random holdout 75/25 ต่อผู้ใช้	`train_r`, `test_r`	ใหม่: split ระดับ interaction
3	Pair features	`PROFILE_FOR_MODEL`, `pair_features`, `build_pair_dataset`, `F_train/F_test`, `yc/yr`	ใหม่ทั้งหมด
4	ฝึกและวัด classifier / regressor	`classifiers`, `regressors`, `cls_rows`, `reg_rows`	ใหม่: Accuracy/Precision/Recall/F1/AUC + MAE/RMSE
5	Predictive ranking + metric บทที่ 2	`best_cls`, `predictive_rank`, `pred_rank_detail`, `summary`	ใหม่: score = P(like)
6	Experiment	`pipeline_results` (Trans × Profile × Model ⟶ AUC)	ใหม่: `clone` โมเดล
7	GUI	`gui_predictive`	แสดง Train History + P(like)

1

Item Vector และ Train/Test Split

Part 1 – Part 2: SBERT + Raw+L2 แล้วแบ่ง rating ของแต่ละคน 75/25

- `X` = SBERT 384 มิติ L2-normalized เป็น space เดียวของ Part 3–5
- `train_r` ใช้สร้างโปรไฟล์และฝึกโมเดล ; `test_r` ใช้วัดผลเท่านั้น

Part 1: SBERT Item Vector

```python
from sentence_transformers import SentenceTransformer
sbert = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
X_raw = sbert.encode(items["text_content"].tolist(),
        show_progress_bar=True)
X = transform_vectors(X_raw, "Raw+L2")
print("Item Vector Shape:", X.shape)
```

อธิบายโค้ด

ทำไมเลือก SBERT

- LAB 3 เน้น pipeline ของโมเดล จึงเลือก representation เดียวที่ให้ dense semantic vector ขนาดพอเหมาะ (384 มิติ) → pair features 769 มิติ ไม่ใหญ่เกินไป
- ถ้าใช้ TF-IDF (หลายร้อยถึงพันมิติ) feature `|p − v|` และ `p ⊙ v` จะมีมิติหลายพันและ sparse → Logistic/RF ฝึกยากกับข้อมูลไม่กี่สิบแถว
- `X_raw` เก็บไว้ → Part 6 ทดลอง `Raw+L2` vs `Center+L2` จาก raw เดียวกัน
- `X` L2 แล้ว → cosine = dot product และ `|p − v|`, `p ⊙ v` อยู่ในสเกลใกล้เคียงกันทุกคู่

Part 2: Random Holdout 75/25 ต่อผู้ใช้

```
rng = np.random.default_rng(42)
train_idx = []; test_idx = []
for uid, g in ratings.groupby("user_id"):
    ids = g.index.to_numpy()
    if len(ids) < 2:
        train_idx += list(ids); continue          # rating เดียว ->
                เข้า train ทั้งหมด
    ntest = max(1, int(round(0.25 * len(ids))))
    chosen = set(rng.choice(ids, size=ntest, replace=False))
    test_idx += list(chosen)
    train_idx += [i for i in ids if i not in chosen]
train_r = ratings.loc[train_idx].copy()
test_r = ratings.loc[test_idx].copy()
print("Train:", len(train_r), "Test:", len(test_r))
```

อธิบายโค้ด

split ระดับ interaction ไม่ใช่ระดับผู้ใช้

- ทุกผู้ใช้อยู่ทั้งใน train และ test (ถ้ามี ≥ 2 rating): **25% ของ rating** ของแต่ละคนถูกซ่อน — ไม่เลือกเฉพาะ positive จึงมีทั้งชอบและไม่ชอบใน test
- ต่างจาก LOO ของ LAB 1–2 ที่ซ่อน positive 1 เรื่อง: test ที่นี่มี label ทั้ง 0 และ 1 ⟶ ใช้ฝึก/วัด classifier และ regressor ได้
- `rng` seed 42 ⟶ ทำซ้ำได้ ; ผู้ใช้ที่มี rating เดียวไม่ถูกทดสอบ
- notebook ระบุว่าถ้ามี timestamp ให้เปลี่ยนเป็น temporal split (ซ่อน 25% ล่าสุด)
- `train_r` เป็น **แหล่งเดียว** ของโปรไฟล์และสถิติทุกอย่างใน Part 3–7 ⟶ held-out ไม่รั่ว

2

User–Item Pair Features

Part 3: เปลี่ยนคู่ (user, item) เป็นเวกเตอร์ feature 769 มิติ

- `pair_features(user_id, item_id, train_df, X)` → `[cos, |p-v|, p⊙v]`
- `build_pair_dataset(df, train_history, X)` → `(F, y_cls, y_reg, meta)`

Part 3: pair_features และ build_pair_dataset

```
PROFILE_FOR_MODEL = "Rating-centered"

def pair_features(user_id, item_id, train_df, X):
    p = get_profile(user_id, X, train_df, PROFILE_FOR_MODEL)     #
            โปรไฟล์จาก train เท่านั้น
    v = X[ITEM_INDEX[str(item_id)]]
    cos = float(np.dot(p, v) / max(np.linalg.norm(p) *
            np.linalg.norm(v), 1e-12))
    return np.concatenate([[cos], np.abs(p - v), p * v])          # 1
            + 384 + 384

def build_pair_dataset(df, train_history, X):
    F = []; y_cls = []; y_reg = []; meta = []
    for _, r in df.iterrows():
        iid = str(r["item_id"]); uid = str(r["user_id"])
        if iid not in ITEM_INDEX: continue
        F.append(pair_features(uid, iid, train_history, X))
        y_reg.append(float(r["rating"]))
        y_cls.append(int(float(r["rating"]) >= 4))
        meta.append((uid, iid))
    return np.asarray(F), np.asarray(y_cls), np.asarray(y_reg), meta

F_train, yc_train, yr_train, meta_train = build_pair_dataset(train_r,
        train_r, X)
F_test,  yc_test,  yr_test,  meta_test  = build_pair_dataset(test_r,
        train_r, X)
print(F_train.shape, F_test.shape)
```

อธิบายโค้ด

feature ที่มากกว่า cosine เดียว

- cos = คะแนน content-based เดิม (1 มิติ) ; |p − v| = ระยะต่อมิติ ; p ⊙ v = ส่วนร่วมต่อมิติของ dot product ⟶ โมเดลถ่วงน้ำหนักได้ว่า **มิติใด** ของ SBERT สำคัญ (cosine ให้น้ำหนักทุกมิติเท่ากัน)
- `PROFILE_FOR_MODEL` เป็น global ที่ `pair_features` อ่าน ⟶ Part 6 เปลี่ยนค่านี้เพื่อทดลอง profile อื่น
- กัน leakage: `build_pair_dataset(test_r, train_r, X)` — feature ของแถว test ใช้โปรไฟล์จาก **train_r** ; แถว train ก็ใช้ train_r (โปรไฟล์จึงมีหนังเรื่องนั้นอยู่ด้วย = optimistic เล็กน้อยใน train ซึ่งยอมรับได้)
- target สองแบบ: `y_cls = rating ≥ 4` (ชอบ/ไม่ชอบ) และ `y_reg = rating` ; `meta` เก็บ (uid, iid) ไว้ตามรอย
- `F_train` มีจำนวนแถว = จำนวน rating ใน train (ไม่กี่สิบ) × 769 คอลัมน์ ⟶ **feature มากกว่าตัวอย่างมาก** จึงต้อง regularize (Logistic C ค่าเริ่มต้น, Ridge α=1, RF)

โครงสร้าง feature ของคู่ (user, item)

ส่วน	สูตร	จำนวนมิติ	ความหมาย
cos	$p \cdot v / (\|p\|\|v\|)$	1	ความคล้ายรวม = คะแนน content-based ของ LAB 1–2
$\|p - v\|$	abs(p − v) ต่อมิติ	384	มิติใดที่โปรไฟล์กับหนังต่างกัน (ยิ่งใกล้ 0 ยิ่งคล้าย)
$p \odot v$	p × v ต่อมิติ	384	มิติใดที่ทั้งคู่ชี้ทางเดียวกัน (บวก) หรือสวนทาง (ลบ)
รวม	concatenate	769	input ของ Logistic / RF / Ridge

ไม่มี user/item statistics (เช่น popularity) ⟶ โมเดลใช้เฉพาะเนื้อหา = ยังเป็น content-based

3

Predictive Models

Part 4: Classification (relevance) และ Regression (rating)

- Logistic Regression / RandomForestClassifier → Accuracy, Precision, Recall, F1, AUC
- Ridge / RandomForestRegressor → MAE, RMSE

Part 4: ฝึกและวัด classifier และ regressor (1/2)

```python
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.ensemble import RandomForestClassifier,
        RandomForestRegressor
from sklearn.metrics import accuracy_score, precision_score,
        recall_score, f1_score, roc_auc_score

classifiers = {
    "LogisticRegression": LogisticRegression(max_iter=2000,
            class_weight="balanced"),
    "RandomForestClassifier": RandomForestClassifier(n_estimators=200,
            random_state=42, class_weight="balanced")}
regressors = {
    "Ridge": Ridge(alpha=1.0),
    "RandomForestRegressor": RandomForestRegressor(n_estimators=200,
            random_state=42)}

cls_rows = []
for name, m in classifiers.items():
    m.fit(F_train, yc_train)
    prob = m.predict_proba(F_test)[:, 1]
```

อธิบายโค้ด

วัดสองงานบน test เดียวกัน

- **Classification:** วัดแบบ pointwise บนคู่ (user, item) ใน test — Accuracy/Precision/Recall/F1 ที่ threshold 0.5 และ AUC จาก probability ; `class_weight="balanced"` ชดเชยถ้า positive มากกว่า negative
- **Regression:** MAE / RMSE ของ rating ที่ทำนาย (`mean_squared_error ** 0.5`)
- โมเดลถูก `fit` ในที่นี้และ **เก็บสถานะไว้** ⟶ `classifiers["LogisticRegression"]` ที่ฝึกแล้วถูกใช้ต่อใน Part 5 และ GUI
- ไม่มี StandardScaler เพราะ feature ทั้ง 769 มิติมาจากเวกเตอร์ L2 อยู่ในสเกลใกล้กัน ; `max_iter=2000` ให้ Logistic ลู่เข้า
- ตัวเลขเหล่านี้ตอบว่า “ทำนายคู่หนึ่งถูกไหม” ยังไม่ใช่ “จัดอันดับให้ผู้ใช้ดีไหม” ⟶ Part 5

Part 4: ฝึกและวัด classifier และ regressor (2/2)

```
    pred = (prob >= 0.5).astype(int)
    cls_rows.append({"Model": name,
                     "Accuracy": accuracy_score(yc_test, pred),
                     "Precision": precision_score(yc_test, pred,
                             zero_division=0),
                     "Recall": recall_score(yc_test, pred,
                             zero_division=0),
                     "F1": f1_score(yc_test, pred, zero_division=0),
                     "AUC": roc_auc_score(yc_test, prob) if
                             len(np.unique(yc_test)) == 2 else np.nan})
display(pd.DataFrame(cls_rows).round(4))

reg_rows = []
for name, m in regressors.items():
    m.fit(F_train, yr_train)
    pred = m.predict(F_test)
    reg_rows.append({"Model": name, "MAE": mean_absolute_error(yr_test,
            pred),
                     "RMSE": mean_squared_error(yr_test, pred) ** 0.5})
display(pd.DataFrame(reg_rows).round(4))
```

อธิบายโค้ด

วัดสองงานบน test เดียวกัน

- **Classification:** วัดแบบ pointwise บนคู่ (user, item) ใน test — Accuracy/Precision/Recall/F1 ที่ threshold 0.5 และ AUC จาก probability ; `class_weight="balanced"` ชดเชยถ้า positive มากกว่า negative
- **Regression:** MAE / RMSE ของ rating ที่ทำนาย (`mean_squared_error ** 0.5`)
- โมเดลถูก `fit` ในที่นี้และ **เก็บสถานะไว้** ⟶ `classifiers["LogisticRegression"]` ที่ฝึกแล้วถูกใช้ต่อใน Part 5 และ GUI
- ไม่มี StandardScaler เพราะ feature ทั้ง 769 มิติมาจากเวกเตอร์ L2 อยู่ในสเกลใกล้กัน ; `max_iter=2000` ให้ Logistic ลู่เข้า
- ตัวเลขเหล่านี้ตอบว่า “ทำนายคู่หนึ่งถูกไหม” ยังไม่ใช่ “จัดอันดับให้ผู้ใช้ดีไหม” ⟶ Part 5

Pointwise metric (Part 4) vs Ranking metric (Part 5)

Part 4 — pointwise

- แต่ละแถว test ถูกทำนายอิสระ ไม่สนว่าเป็นผู้ใช้คนไหน
- Accuracy/F1 ขึ้นกับ threshold 0.5 ; AUC ไม่ขึ้นกับ threshold
- MAE/RMSE บอกความคลาดเคลื่อนของ rating แต่ **ไม่บอกว่า Top-K ถูกไหม**

Part 5 — ranking

- จัดอันดับ candidate ทั้งหมดของผู้ใช้ด้วย `P(like)` แล้วดูว่า relevant (rating ≥ 4 ใน test) อยู่ Top-3 ไหม
- ใช้ metric บทที่ 2 ทั้ง Accuracy/Ranking และ Beyond-Accuracy ⟶ เทียบกับ LAB 1–2 ได้
- AUC ดีใน Part 4 ≠ nDCG ดีใน Part 5 เสมอไป

LAB 3 จงใจแสดงทั้งสองมุมเพื่อให้เห็นว่า metric ต่างระดับตอบคำถามต่างกัน

4

Predictive Ranking และ Metric บทที่ 2

Part 5: ใช้ P(like) ของ Logistic Regression เป็นคะแนนจัดอันดับ

- `predictive_rank(uid, model, X, train_df)` ⟶ `(ranked, score dict)`
- relevant = เรื่องใน test ที่ rating ≥ 4 (อาจมีหลายเรื่องต่อคน)

Part 5: predictive_rank และ loop ประเมินต่อผู้ใช้ (1/2)

```python
best_cls = classifiers["LogisticRegression"]

def predictive_rank(uid, model, X, train_df):
    seen = set(train_df.loc[train_df["user_id"].astype(str) ==
            str(uid), "item_id"].astype(str))
    cand = [iid for iid in items["item_id"] if iid not in seen]
    if not cand: return [], {}
    F = np.vstack([pair_features(uid, iid, train_df, X) for iid in
            cand])
    prob = model.predict_proba(F)[:, 1]
    score = dict(zip(cand, prob))
    return sorted(cand, key=lambda x: score[x], reverse=True), score

rows = []; all_rec = []
for uid, g in test_r.groupby("user_id"):
    rel = set(g.loc[g["rating"] >= 4, "item_id"].astype(str))
    if not rel: continue
    ranked, score = predictive_rank(str(uid), best_cls, X, train_r)
    top3 = ranked[:3]; all_rec += top3
```

อธิบายโค้ด

loop ประเมินที่เขียนใหม่ (ไม่ใช้ evaluate_config)

- `predictive_rank`: candidate = ทุกเรื่องที่ไม่อยู่ใน train ของผู้ใช้ (รวมเรื่องใน test ทั้งชอบและไม่ชอบ) ⟶ สร้าง pair features ⟶ `predict_proba` ⟶ เรียงตาม P(like)
- **relevant มีหลายเรื่อง** (test ที่ rating ≥ 4) ⟶ Precision@3, Recall@3, MAP@3 ให้ค่าที่ต่างจาก HR ต่างจาก LOO ที่ relevant เดียว
- ผู้ใช้ที่ test ไม่มี positive ถูกข้าม ; ใช้ metric ฟังก์ชันเดิมจาก Core ทั้งหมด ⟶ เทียบกับ `lab1_results/experiment_results` ได้ (แต่ split ต่างกัน ต้องระบุ)
- Beyond-Accuracy: ILD ใน space SBERT, Novelty, Calibration เทียบกับ `train_r`, Coverage/Gini จาก `all_rec`
- `best_cls` = Logistic ที่ฝึกแล้วใน Part 4 ; เปลี่ยนเป็น RF ได้บรรทัดเดียว

Part 5: predictive_rank และ loop ประเมินต่อผู้ใช้ (2/2)

```
    rows.append({"user_id": uid,
                 "Precision@3": precision_at_k(top3, rel, 3),
                        "Recall@3": recall_at_k(top3, rel, 3),
                 "F1@3": f1_at_k(top3, rel, 3), "HR@3": hr_at_k(top3,
                        rel, 3),
                 "MRR": reciprocal_rank(ranked, rel), "MAP@3":
                        ap_at_k(top3, rel, 3),
                 "nDCG@3": ndcg_at_k(top3, rel, 3), "ILD": ild(top3,
                        X), "Novelty": novelty(top3),
                 "Calibration": calibration_genre(str(uid), top3,
                        train_r)})
pred_rank_detail = pd.DataFrame(rows)
summary = pred_rank_detail.select_dtypes(include=np.number).mean().to_di
        ct()
summary["Coverage"] = coverage(all_rec); summary["Gini"] =
        gini_coefficient(all_rec)
display(pred_rank_detail.round(4));
        display(pd.Series(summary).to_frame("mean").round(4))
```

อธิบายโค้ด

loop ประเมินที่เขียนใหม่ (ไม่ใช้ evaluate_config)

- `predictive_rank`: candidate = ทุกเรื่องที่ไม่อยู่ใน train ของผู้ใช้ (รวมเรื่องใน test ทั้งชอบและไม่ชอบ) ⟶ สร้าง pair features ⟶ `predict_proba` ⟶ เรียงตาม P(like)
- relevant **มีหลายเรื่อง** (test ที่ rating ≥ 4) ⟶ Precision@3, Recall@3, MAP@3 ให้ค่าที่ต่างจาก HR ต่างจาก LOO ที่ relevant เดียว
- ผู้ใช้ที่ test ไม่มี positive ถูกข้าม ; ใช้ metric ฟังก์ชันเดิมจาก Core ทั้งหมด ⟶ เทียบกับ `lab1_results/experiment_results` ได้ (แต่ split ต่างกัน ต้องระบุ)
- Beyond-Accuracy: ILD ใน space SBERT, Novelty, Calibration เทียบกับ `train_r`, Coverage/Gini จาก `all_rec`
- `best_cls` = Logistic ที่ฝึกแล้วใน Part 4 ; เปลี่ยนเป็น RF ได้บรรทัดเดียว

จากคู่ (user, item) สู่ Top-K ด้วยโมเดล (Part 3–5)

train_r ของผู้ใช้ ไม่มี rating ใน test

↓

p = get_profile(uid, X, train_r, Rating-centered)

↓

candidate = unseen ใน train รวมเรื่องใน test

↓

pair_features ต่อ candidate [cos, |p−v|, p ⊙ v]

↓

model.predict_proba ⟶ P(like) Logistic Regression

↓

sorted ⟶ Top-3 predictive_rank

↓

relevant (test, rating ≥ 4) อยู่อันดับใด? P R F1 HR MRR MAP nDCG + Beyond

เทียบกับ LAB 1–2

- LAB 1–2: score = cosine(p, v) ค่าเดียว ; LAB 3: score = $\sigma(w \cdot [\cos, |p-v|, p \odot v] + b)$ ที่ **เรียนรู้ w** จาก train
- ถ้า w ให้น้ำหนักเฉพาะ cos ⟶ กลับไปเท่ากับ LAB 1 ; feature ต่อมิติทำให้โมเดลชดเชยมิติที่ไม่สำคัญได้
- ข้อมูลไม่กี่สิบแถว × 769 feature ⟶ เสี่ยง overfit ; ดู AUC ใน Part 6 ประกอบ

5

Experiment และ GUI

Part 6 – Part 7: Transformation × Profile × Model แล้วสาธิตด้วย Gradio

- `clone` โมเดลเพื่อฝึกใหม่ต่อเงื่อนไข ; วัดด้วย AUC บน test
- GUI แสดง Train History และ Predictive Recommendations พร้อม P(like)

Part 6: Transformation × Profile × Model ⟶ AUC

```
from sklearn.base import clone
pipeline_rows = []
for trans in ["Raw+L2", "Center+L2"]:
    Xt = transform_vectors(X_raw, trans)
    for prof in ["Positive-only", "Rating-centered", "Rocchio"]:
        PROFILE_FOR_MODEL = prof                          # pair_features
                อ่าน global นี้
        Ft, yct, _, _ = build_pair_dataset(train_r, train_r, Xt)
        Fv, ycv, _, _ = build_pair_dataset(test_r, train_r, Xt)
        for name, base_model in classifiers.items():
            m = clone(base_model); m.fit(Ft, yct)
            prob = m.predict_proba(Fv)[:, 1]
            pipeline_rows.append({"Transformation": trans, "Profile":
                    prof, "Model": name,
                                  "AUC": roc_auc_score(ycv, prob) if
                                         len(np.unique(ycv)) == 2 else
                                                np.nan})
pipeline_results = pd.DataFrame(pipeline_rows)
display(pipeline_results.round(4))
```

อธิบายโค้ด

แยกผลของ representation ออกจาก algorithm

- split เดิม (`train_r`/`test_r`) ทุกเงื่อนไข ⟶ ผลต่างของ AUC มาจาก transformation / profile / model เท่านั้น
- เปลี่ยน `PROFILE_FOR_MODEL` (global) แล้วสร้าง pair dataset ใหม่ทั้ง train และ test ต่อรอบ — เพราะ feature ขึ้นกับโปรไฟล์
- `clone(base_model)` = โมเดลใหม่ที่ยังไม่ฝึก พารามิเตอร์เดิม ⟶ ไม่ปนสถานะกับ `classifiers` ที่ฝึกใน Part 4 (ซึ่ง GUI ยังใช้อยู่)
- 2 × 3 × 2 = 12 แถว วัดด้วย AUC pointwise (ไม่ใช่ ranking) — ต้องการ ranking metric ให้ใส่ loop ของ Part 5 เข้าไปในแต่ละรอบ
- **หมายเหตุ:** หลัง loop `PROFILE_FOR_MODEL` ค้างเป็น `"Rocchio"` ⟶ Part 7 ตั้งกลับเป็น `"Rating-centered"` ก่อนใช้ `best_cls` ที่ฝึกด้วย profile นั้น

Part 7: gui_predictive และ gr.Interface (1/2)

```
import gradio as gr
PROFILE_FOR_MODEL = "Rating-centered"      # ต้องตรงกับ profile
        ที่ใช้ฝึก best_cls

def gui_predictive(user_id, k):
    ranked, score = predictive_rank(str(user_id), best_cls, X, train_r)
    top = ranked[:int(k)]
    hist = train_r[train_r["user_id"].astype(str) ==
            str(user_id)].merge(
        items[["item_id", "title", "genre"]], on="item_id", how="left"
    )[["title", "genre", "rating"]].sort_values("rating",
            ascending=False)
    rec = []
    for rank, iid in enumerate(top, 1):
        r = items.iloc[ITEM_INDEX[iid]]
```

อธิบายโค้ด

GUI ที่แสดง “ความน่าจะเป็น” แทน cosine

- ใช้ `predictive_rank` กับ `train_r` (ไม่ใช่ `ratings` เต็ม) ⟶ คำแนะนำอาจรวมเรื่องที่อยู่ใน test ของผู้ใช้ — เปิดโอกาสให้เทียบว่าโมเดลทายเรื่องที่ซ่อนไว้ได้ไหม
- คอลัมน์ **P(like)** ∈ [0, 1] จาก Logistic Regression — ความหมายต่างจาก Score (cosine) ของ LAB 1–2
- แสดง **Train History** เพื่อให้เห็นว่าโปรไฟล์สร้างจากอะไร ; ไม่มี Profile Terms เพราะ SBERT ไม่มีชื่อมิติ
- input เหลือแค่ User และ Top-K เพราะ representation/profile/model ถูกตรึงไว้ที่ SBERT + Rating-centered + Logistic
- แนวทางสอน: เลือกผู้ใช้ ⟶ ดู Train History ⟶ ดู Top-K พร้อม P(like) ⟶ เทียบกับ rating จริงใน `test_r` ของผู้ใช้คนนั้น

Part 7: gui_predictive และ gr.Interface (2/2)

```
        rec.append({"Rank": rank, "Title": r["title"], "Genre":
                r["genre"],
                    "P(like)": round(float(score[iid]), 4), "Synopsis":
                            r["synopsis"]})
    return hist, pd.DataFrame(rec)

demo = gr.Interface(
    fn=gui_predictive,
    inputs=[gr.Dropdown(sorted(ratings["user_id"].unique()),
            label="User"),
            gr.Slider(1, 5, 3, 1, label="Top-K")],
    outputs=[gr.Dataframe(label="Train History"),
            gr.Dataframe(label="Predictive Recommendations")],
    title="Chapter 3 – End-to-End Predictive Content Recommender")
demo.launch(debug=False)
```

อธิบายโค้ด

GUI ที่แสดง “ความน่าจะเป็น” แทน cosine

- ใช้ `predictive_rank` กับ `train_r` (ไม่ใช่ `ratings` เต็ม) ⟶ คำแนะนำอาจรวมเรื่องที่อยู่ใน test ของผู้ใช้ — เปิดโอกาสให้เทียบว่าโมเดลทายเรื่องที่ซ่อนไว้ได้ไหม
- คอลัมน์ **P(like)** $\in$ [0, 1] จาก Logistic Regression — ความหมายต่างจาก Score (cosine) ของ LAB 1–2
- แสดง **Train History** เพื่อให้เห็นว่าโปรไฟล์สร้างจากอะไร ; ไม่มี Profile Terms เพราะ SBERT ไม่มีชื่อมิติ
- input เหลือแค่ User และ Top-K เพราะ representation/profile/model ถูกตรึงไว้ที่ SBERT + Rating-centered + Logistic
- แนวทางสอน: เลือกผู้ใช้ ⟶ ดู Train History ⟶ ดู Top-K พร้อม P(like) ⟶ เทียบกับ rating จริงใน `test_r` ของผู้ใช้คนนั้น

สรุปแนวคิดของ Source Code LAB 3

Take-away

- **Core เดิม + pipeline ใหม่:** ใช้ `get_profile`, `transform_vectors`, metric จาก LAB 1 แต่แทน `evaluate_config` ด้วย loop ที่จัดอันดับด้วยโมเดล
- **Split ระดับ interaction (75/25 ต่อผู้ใช้):** test มีทั้ง label 0/1 และ rating จริง ⟶ ฝึก/วัด classifier กับ regressor ได้ และ relevant ต่อคนมีหลายเรื่อง
- **Pair features = [cos, |p−v|, p⊙v] (769 มิติ):** ให้โมเดลถ่วงน้ำหนักต่อมิติของ SBERT แทน cosine ที่ให้น้ำหนักเท่ากัน ; โปรไฟล์มาจาก `train_r` เสมอ
- **สองระดับของการวัด:** pointwise (Accuracy/F1/AUC, MAE/RMSE) ใน Part 4 และ ranking ต่อผู้ใช้ (P/R/F1/HR/MRR/MAP/nDCG + Beyond) ใน Part 5 — ดีในระดับหนึ่งไม่รับประกันอีกระดับ
- **`PROFILE_FOR_MODEL` เป็น global** ที่ pair_features อ่าน ⟶ Part 6 สลับ profile ได้ แต่ต้องตั้งกลับให้ตรงกับโมเดลที่ GUI ใช้
- **`clone` + split เดิม** ทำให้ Part 6 แยกผลของ representation/profile ออกจาก algorithm ได้ ; ข้อมูลไม่กี่สิบแถว × 769 feature ⟶ ตีความอย่างระวัง

Data ⟶ Item Vector ⟶ Profile ⟶ Pair Features ⟶ Model ⟶ Ranking ⟶ Evaluation ⟶ GUI